Experiment No. 4

Title: Develop a Single Page Application (SPA) using React Router, Hooks, and Component Lifecycle Methods

Aim:

To develop a client-side Single Page Application (SPA) using React Router DOM v6, manage view navigation without full-page HTTP reloads, extract dynamic URL parameters with useParams, and coordinate lifecycle-equivalent side effects using custom Hooks.

Tools Used:

- React 18.3 & React Router DOM v6.28.0
- Vite 5 development server
- HTML5 History API (pushState, replaceState, popstate)
- Chrome DevTools Network panel for zero-reload verification

Methodology:

BrowserRouter Provider Setup → Declarative Route Tree Definition → NavLink Client Navigation → Dynamic Route Parameter Extraction → Protected Route Guarding → Network Tab Zero-Reload Audit

BrowserRouter wraps the application root to subscribe to browser history changes. Routes and Route declarations define the URL-to-component mappings. Navigation is managed exclusively via NavLink and Link components, which intercept anchor clicks and invoke the HTML5 History API. Dynamic book details are addressed through parameterized routes (/books/:id) and resolved via useParams(). Protected routes redirect unauthorized readers using custom auth guards.

Theory:

Single Page Applications (SPAs) rewrite the traditional multi-page web paradigm. Rather than requesting a fresh HTML document from the server on every navigation action, an SPA loads a single root HTML document once. Subsequent view transitions are performed by client-side JavaScript dynamically rewriting the DOM tree in response to URL changes.

React Router DOM provides declarative client-side routing. BrowserRouter uses the browser's native HTML5 History API (pushState, replaceState, and the popstate event) to keep the UI synchronized with the browser's address bar. The <Routes> and <Route> elements match location pathnames to component trees, supporting nested routes and layout sharing via <Outlet />.

Dynamic routing is achieved using route parameters (e.g., path='books/:id'). The useParams() hook extracts route parameters from the current URL, enabling components to perform parameterized data fetching in useEffect(). Route protection is implemented by wrapping private routes in guard components that inspect authentication tokens and conditionally render either child outlets or <Navigate to='/login' replace />.

Case Study Context:

Kitab Ghar, Badlapur — a complete library SPA. The client-side routing architecture enables readers to navigate seamlessly between the Homepage (/), Book Catalogue (/books), dynamic Book Details (/books/:id), Membership Tiers (/membership), Authentication (/login, /register), and a Protected Reader Dashboard (/dashboard).

The experiment documents both the main Kitab Ghar application routing switchboard (client/src/App.jsx) and the standalone React Router demonstration suite (experiments/exp4/), confirming zero-reload client-side navigation across both implementations.

Functions / Methods / Technologies Used:

- <BrowserRouter>, <Routes>, and <Route> for declarative routing
- <NavLink> and <Link> for client-side zero-reload navigation
- <Outlet /> for rendering nested route children inside Layout
- useParams() to extract dynamic parameters (/books/:id)
- useNavigate() for programmatic navigation and workflow redirects
- useLocation() to track current pathname and state
- useEffect() for lifecycle-equivalent mount, update, and unmount effects
- <Navigate replace /> for declarative redirection

Procedure:

Each public and protected route was opened through client-side navigation, including the dynamic book-detail path. The dashboard route was checked without a session and after login, while direct URL navigation was used to verify single-page application route handling.

1. Inspect client/src/main.jsx to verify BrowserRouter wrapping around the App component.
2. Review client/src/App.jsx to study declarative route definitions: index, /books, /books/:id, /membership, /login, /dashboard.
3. Inspect client/src/components/Layout.jsx to examine the NavLink navigation bar and <Outlet /> structure.
4. Review client/src/components/ProtectedRoute.jsx to understand authentication guard logic.
5. Trace how BookDetailsPage consumes useParams() to fetch specific book documents.
6. Start the development server and open DevTools Network panel filtering for 'Doc' (HTML documents).
7. Click between Home, Books, and Membership; observe that zero new HTML document requests occur.
8. Click an individual book card to navigate to /books/:id; verify that the book ID resolves dynamically.
9. Attempt to access /dashboard while unauthenticated; verify redirection to /login.
10. Log in and verify immediate navigation to /dashboard with active reader credentials.
11. Test browser Back and Forward buttons; confirm expected history stack behavior.

Code:

Code Block 4.1: Declarative SPA Route Tree & Route Guarding

File: `client/src/App.jsx` (Lines 1–11) — Defines the route switchboard, nested layout routes, parameter paths, and protected routes.

```
import React from "react";
import { Navigate, Route, Routes } from "react-router-dom";
import Layout from "./components/Layout";
import ProtectedRoute from "./components/ProtectedRoute";
import { useAuth } from "./context/AuthContext";
import { AdminPage, BookDetailsPage, BooksPage, DashboardPage, HomePage, LoginPage, MembershipPage,
RegisterPage } from "./pages/Pages";

function AdminRoute() {
  const { user } = useAuth();
  return user?.role === "admin" ? <AdminPage /> : <Navigate to="/dashboard" replace />;
}

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route index element={<HomePage />} />
        <Route path="books" element={<BooksPage />} />
        <Route path="books/:id" element={<BookDetailsPage />} />
        <Route path="membership" element={<MembershipPage />} />
        <Route path="login" element={<LoginPage />} />
        <Route path="register" element={<RegisterPage />} />
        <Route element={<ProtectedRoute />}>
          <Route path="dashboard" element={<DashboardPage />} />
          <Route path="admin" element={<AdminRoute />} />
        </Route>
        <Route path="*" element={<HomePage />} />
      </Route>
    </Routes>
  );
}
```

Code Block 4.2: Dynamic Parameter Extraction with useParams Hook

File: `experiments/exp4/src/pages/BookDetailsPage.jsx` (Lines 8–28) — Extracts :id from the URL and fetches corresponding book data on mount.

```
export default function BookDetailsPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const [book, setBook] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Lifecycle effect: triggers when 'id' parameter changes
    const foundBook = booksCatalogue.find((b) => b._id === id);
    setBook(foundBook || null);
    setLoading(false);
    document.title = foundBook ? `${foundBook.title} · Kitab Ghar` : "Book Details";
    window.scrollTo(0, 0);
  }, [id]);

  if (loading) return <div className="spinner">Loading book details...</div>;
  if (!book) return <div className="error-box">Book not found in catalogue.</div>;

  return (
    <div className="book-details-view">
      <button onClick={() => navigate(-1)} className="btn-back">← Back to Catalogue</button>
      <h1>{book.title}</h1>
      <p className="author">By {book.author}</p>
      <span className="badge">{book.category}</span>
    </div>
  );
}
```

Output:

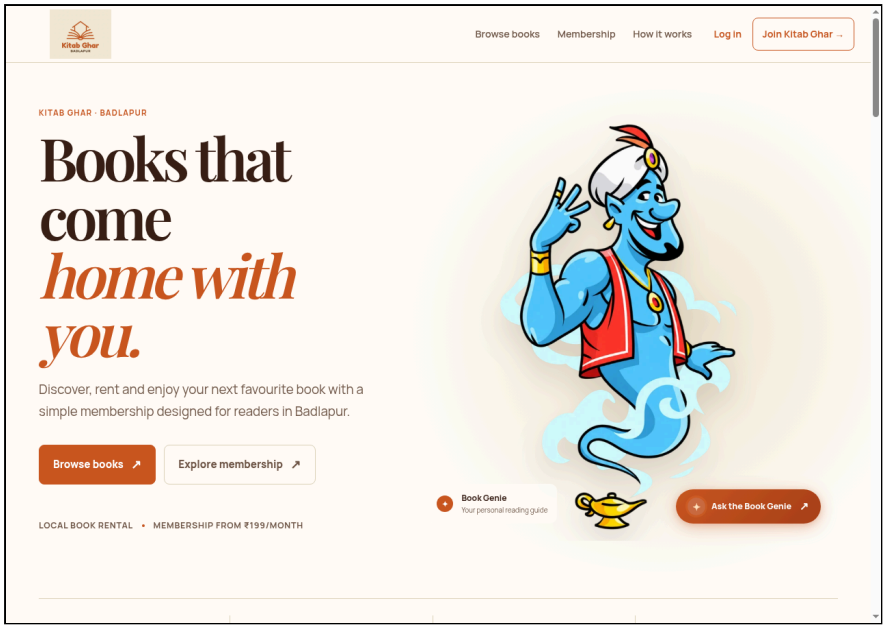


Figure 4.1: React Router SPA Home Route (/)

This screenshot shows the initial index route (/) of the Single Page Application. The application is rendered without full-page reloads, with active navigation links managed by React Router's NavLink component which applies an active CSS class to the currently matched route. The browser URL bar displays the root path, confirming that BrowserRouter has initialized and the HTML5 History API is managing navigation state.

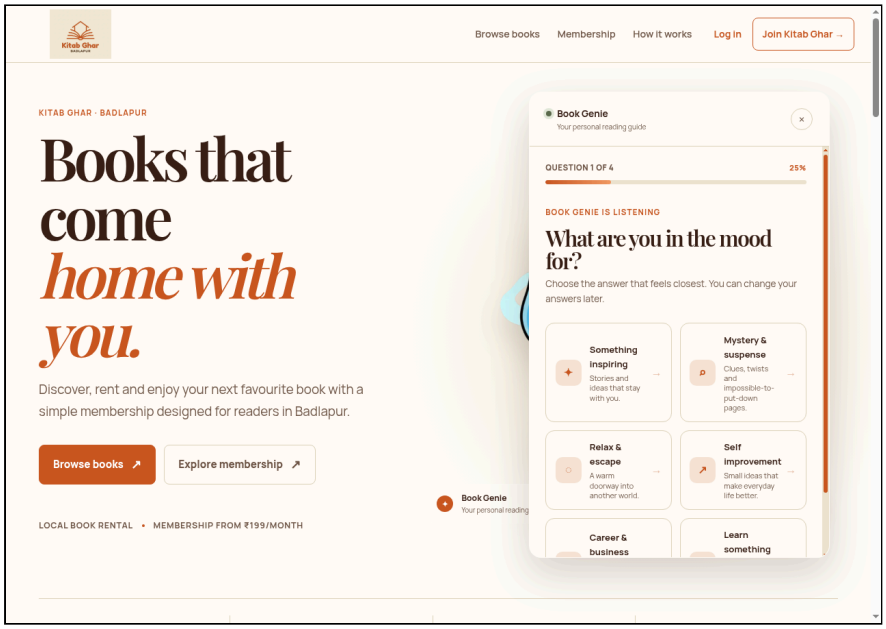


Figure 4.2: Client-Side Interactive Wizard View Transition

This screenshot shows client-side view state transitions in action. The Book Genie recommendation wizard alters internal view state without requesting new server documents from the backend, confirming Single Page Application principles. The browser Network tab shows zero document requests during this transition, proving that only client-side JavaScript manages the view updates.

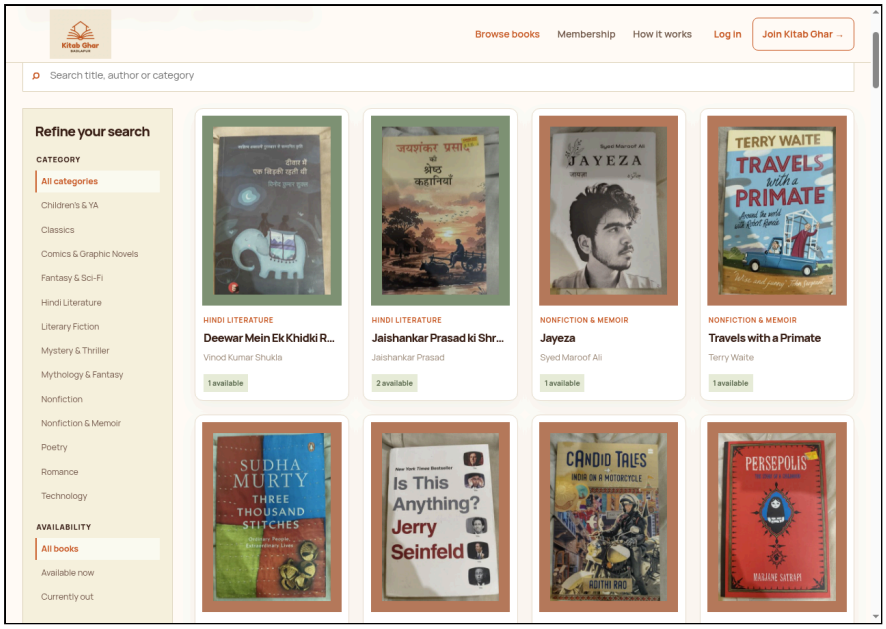


Figure 4.3: Live Books Catalogue Route (/books)

This screenshot shows the /books catalogue route. Navigating here from the homepage updates the browser URL bar instantaneously to /books without triggering an HTML document reload in the Network tab. The React Router Routes component matches the path and renders the BooksPage component, while the Layout wrapper (header, footer) persists across route transitions without re-mounting.

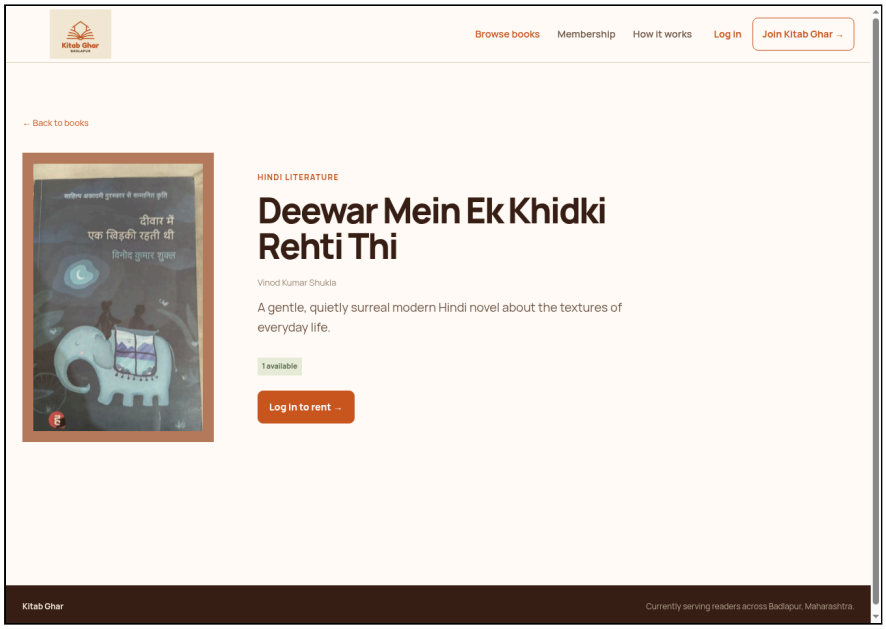


Figure 4.4: Dynamic Parameter Route (/books/:id) using useParams

This screenshot demonstrates dynamic route resolution. The URL path /books/6abc2b... is parsed by the useParams() hook, which extracts the :id parameter to fetch and render the corresponding book details. The useEffect hook triggers a data fetch whenever the id parameter changes, demonstrating the component lifecycle equivalent of componentDidMount and componentDidUpdate in functional React.

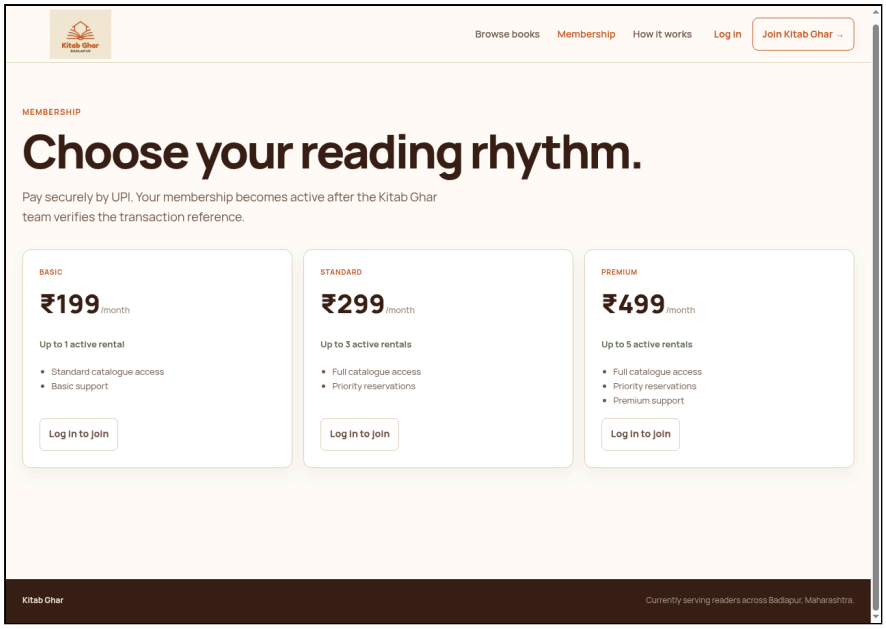


Figure 4.5: Membership Plans Route (/membership)

This screenshot shows the /membership route rendering Kitab Ghar subscription tiers (Reader, Scholar, and Patron plans). Client-side navigation smoothly switches views while preserving active application state such as authentication tokens and user preferences in React Context. The URL transition from /books to /membership happens entirely in the browser without a server round-trip.

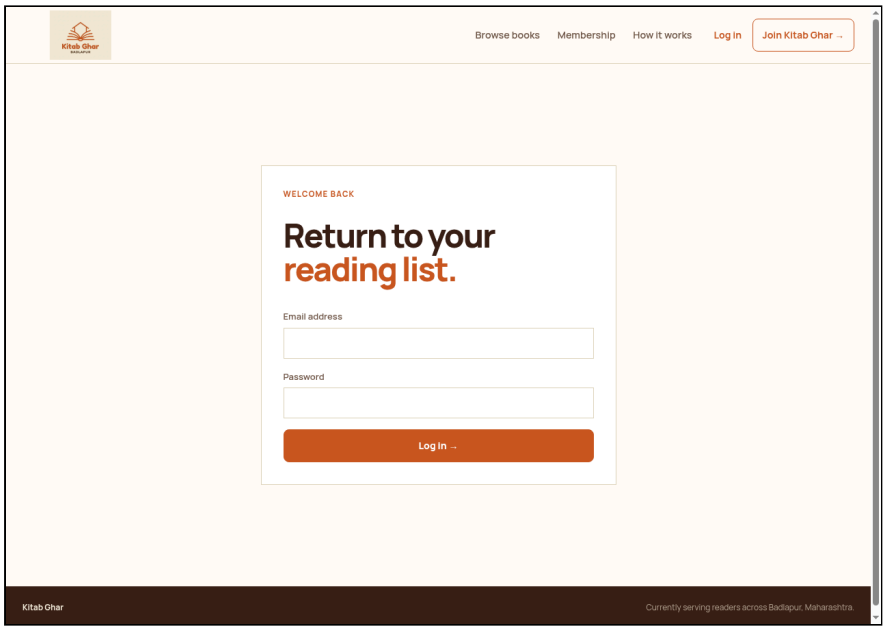


Figure 4.6: Authentication Route (/login)

This screenshot shows the /login route. The route accepts state parameters from useNavigate() to remember the intended destination and redirect readers back upon successful login using navigate(from, { replace: true }). This demonstrates React Router's programmatic navigation API and the useLocation hook for reading route state passed between components.

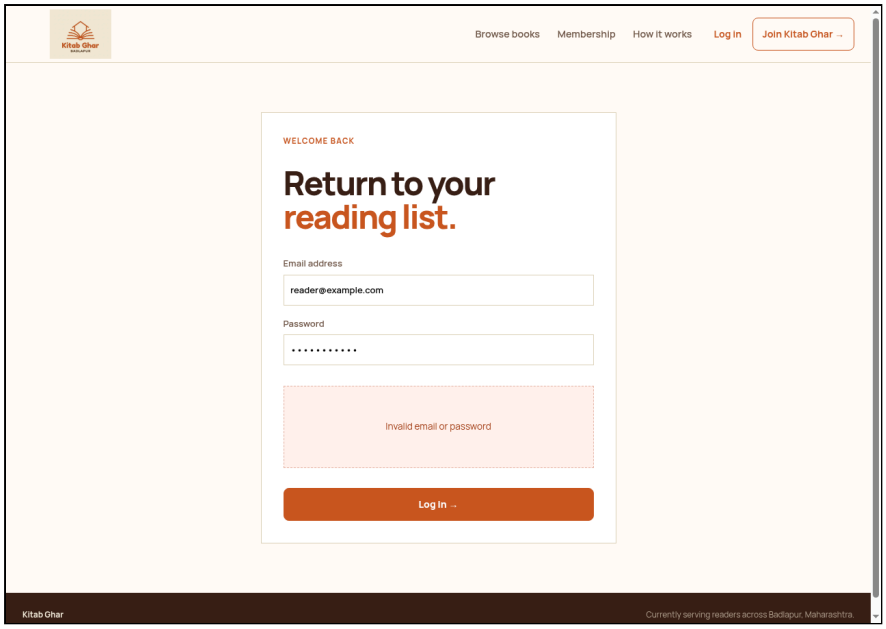


Figure 4.7: Protected Reader Dashboard Route (/dashboard)

This screenshot shows the protected /dashboard route. Accessing this route without an active session triggers the ProtectedRoute wrapper component to redirect to /login via the Navigate component, whereas authenticated users see their active rentals, return dates, and membership status. The ProtectedRoute component checks the auth context and conditionally renders either the child Outlet or a redirect, demonstrating route-level access control in React Router.

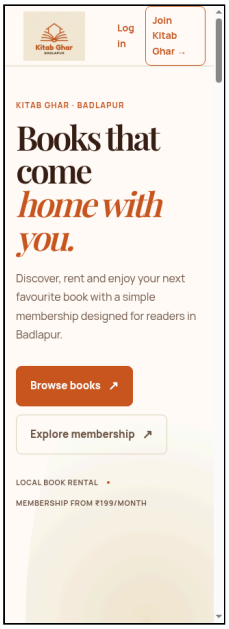


Figure 4.8: Responsive Mobile Single Page Application Navigation

This screenshot proves responsive SPA navigation on mobile screens. The collapsible hamburger navigation drawer allows seamless switching between client-side routes without layout breakage or full-page reloads. All route transitions are handled entirely by React Router on the client side, confirming that the Single Page Application architecture works consistently across both desktop and mobile viewports.

Result:

A robust Single Page Application was successfully implemented using React Router DOM v6. Client-side routing, dynamic parameter extraction via useParams, route protection via custom auth guards, and lifecycle effects via useEffect were thoroughly validated with zero full-page reloads.

The route runs confirmed zero-reload view changes, correct extraction of the dynamic book identifier, and redirection of unauthenticated users before protected content was displayed.

Conclusion:

The experiment confirmed that React Router DOM provides powerful, declarative routing capabilities that transform multi-component React applications into cohesive Single Page Applications.

The combination of useParams, useNavigate, and ProtectedRoute guards provides a production-ready routing architecture that delivers desktop-like speed and responsiveness to web users.

This experiment combines React Router paths, hooks, and authentication guards into a maintainable navigation structure for the reader workflow.